

Informe de consultoría

Proyecto: Predicción de costos operativos del proyecto de construcción

Consultor responsable: Dennis Andres Forero – Data Scientist

Explicación del caso

EL actual proyecto nace como respuesta a la necesidad del negocio de crear un presupuesto de costos operativos más certero y cercano a la realidad, que contemple la incertidumbre y saque provecho de la información que se esconde en comportamientos históricos, patrones y relaciones entre factores que impactan el comportamiento del precio de componentes y equipos usados en la operación.

Se trata de un problema clásico en el ámbito de cadena de suministros y planeación de operaciones, donde variables como stock, lead-time, planeación presupuestal y el costo operativo se convierten en indicadores vitales para que, en el presente, se tomen decisiones que ayudarán a que la operación no se interrumpa en el futuro.

Se debe tener en cuenta que nos enfrentamos a un problema donde el insumo para resolverlo son datos de naturaleza secuencial y, por lo tanto, tienen una jerarquía en el tiempo. Es por eso que se debe definir qué información tenemos del pasado y del presente que nos permita analizar y crear un proceso de búsqueda de un modelo basado en modelos estadísticos y de aprendizaje automático, que se valide en información histórica, simulando su desempeño en un escenario real si se viajara en el tiempo a resolver el mismo problema, con la diferencia de que sí contamos con datos reales para evaluar qué tan bueno o malo es un modelo, y desplegar en el ámbito productivo el modelo que reportó mayor poder predictivo.

Supuestos

El enunciado del problema nos obliga a tomar decisiones basadas en limitaciones en la información y supuestos con fines conceptuales, técnicos y prácticos.

1. Se asume **2023-08-31** como la fecha presente. El último día donde tuvimos información sobre el comportamiento de las diferentes variables de interés para el problema.
2. **El precio de un equipo no está disponible antes que otro.** En otras palabras, no se puede usar el precio de un equipo como un factor o feature para predecir el precio del otro.
3. Los días en los que no se encuentran registros de precios, que pueden ser vacíos de hasta 4 días, **se supone que hay una relación lineal entre el último valor conocido y el siguiente valor conocido** con fines de imputación y de creación de una base histórica continua. Pues el pronóstico futuro también debe ser continuo. Una alternativa válida a esta suposición puede ser usar el último valor conocido, pues los saltos representan que no hubo cambios en los precios. Pero para fines prácticos se supone el primer escenario.
4. Se asume que el **horizonte de tiempo para la predicción se solicita en la frecuencia diaria**, y que el forecast se debe entregar en esta misma frecuencia sin ningún tipo de agregaciones a otra escala de tiempo.
5. Se supone que **las variables de costos de los insumos X, Y, Z tampoco están disponibles en su valor real posterior a 2023-08-31**, a pesar de que sus archivos .csv sí tengan, en ciertos casos, información posterior. Esta suposición se debe a que, en la práctica, tener esta información no es común, excepto en casos donde las fechas de corte o ingesta de los datos crean espacios en la disponibilidad de estos. Sin embargo, por lo general, estos factores tienen por sí mismos su comportamiento variable y con incertidumbre intrínseca de cara al futuro.

Formas para resolver el caso y la opción tomada en esta prueba

1. **Alternativa A:** Usar un modelo estadístico o de regresión lineal para cada target, usando como features los valores lagados de los insumos de mayor correlación lineal.
2. **Alternativa B:** Usar un modelo global de regresión basado en árboles que se entrene con datos de ambos targets, lags de los precios de los insumos y componentes basados en estacionalidades, usando una estrategia de validación de cross-validation para seleccionar el mejor modelo.

Se usará una parametrización para definir el número de pasos en días que se quiere ver hacia el futuro y la métrica error a minimizar y, con esta información, realizar el proceso de evaluación y selección del mejor modelo. Esto con el fin de dar libertad a diversas necesidades de planeación, optimizar el mejor pronóstico posible con esa ventana y también debido a que el enunciado no da información sobre la ventana en que el equipo de planeación y presupuesto debe tomar decisiones para el futuro.

3. **Opción seleccionada:** La opción usada es la **B**, pues es una alternativa que buscará capturar relaciones lineales, pero también otras no lineales que se pueden escapar de modelos estadísticos y de regresión múltiple, pues los modelos basados en árboles son capaces de capturar relaciones más complejas que, según el EDA, se comprueba que están presentes en los datos, más allá de la alta colinealidad entre precios de equipo y de algunos insumos.

Resultados del análisis de: Datos

Para la ejecución al corte de **2023-08-31**, se inicia con un proceso exploratorio automático de datos o EDA que busca computar métricas numéricas sobre tendencia central de las variables y otras métricas de dispersión o de análisis de distribución, tanto de forma univariada como comparando múltiples variables.

Histórico de equipos

Resumen cuantitativo de las variables disponibles en esta fuente, con **3530** registros y 0 filas con valores nulos o NAN.

Metric	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2
count	3,530.0	3,530.0	3,530.0	3,530.0	3,530.0
mean	78.1	555.5	2,037.4	460.0	890.0
std	25.2	138.5	373.1	113.7	170.0
min	19.3	257.5	1,421.5	208.3	566.0
25 %	57.1	482.5	1,767.3	398.2	777.7
50 %	75.4	541.625,00	1,974.8	451.3	869.785,00
75 %	104.6	620.0	2,235.9	515.6	979.1
max	128.0	1,062.4	3,984.0	855.3	1,704.0

Figura 1. Tabla de estadística descriptiva en variables numéricas para historico_equipos.csv

En cuanto a la columna Date, se encontró que:

Min Date: 2010-01-04 00:00:00

Max Date: 2023-08-31 00:00:00

Date difference: 13 years, 8 months, 2 days

Infered frequency: None

La frecuencia inferida None es un indicador de que hay saltos temporales, pues no se encontró un comportamiento continuo con el mismo número de tiempo entre registros. Al graficar un segmento de la histórica disponible, se observan estos saltos:

Salto	#registros
1 days	2808
3 days	687
4 days	25
2 days	9

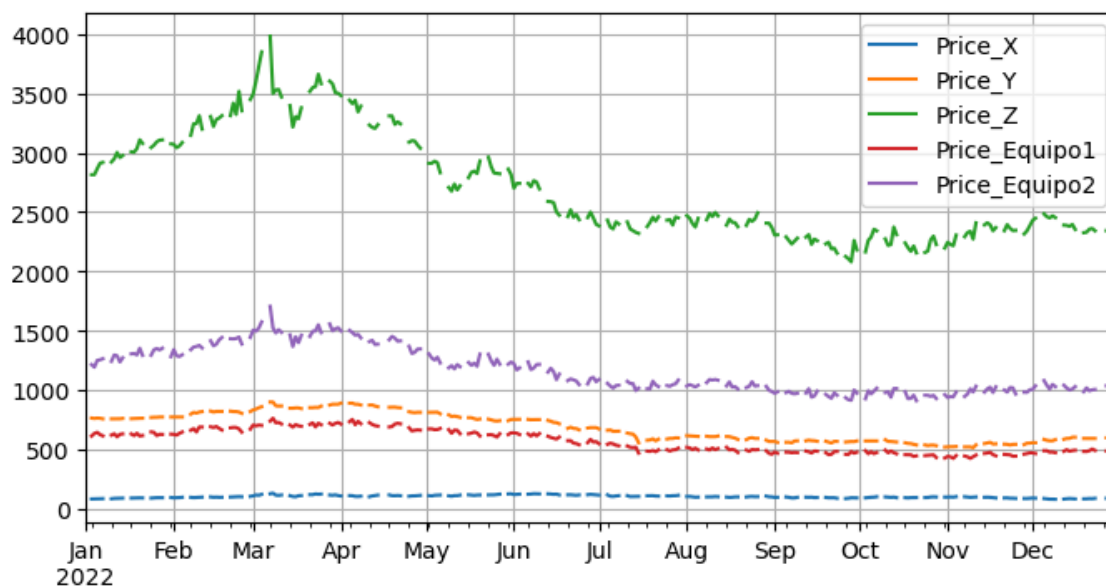


Figura 2. Gráfica de 2022 que demuestra saltos temporales o no disponibilidad de datos en todas las variables.

En este caso se aplica la suposición número 3, y supondremos que podemos imputar estos espacios con una interpolación lineal, con base en un análisis visual del comportamiento de los datos y de los saltos, donde, a pesar de haber cambios bruscos, se puede trazar una trayectoria lineal entre cada punto y no como una función salto o donde un valor permanece igual por varios periodos de tiempo.

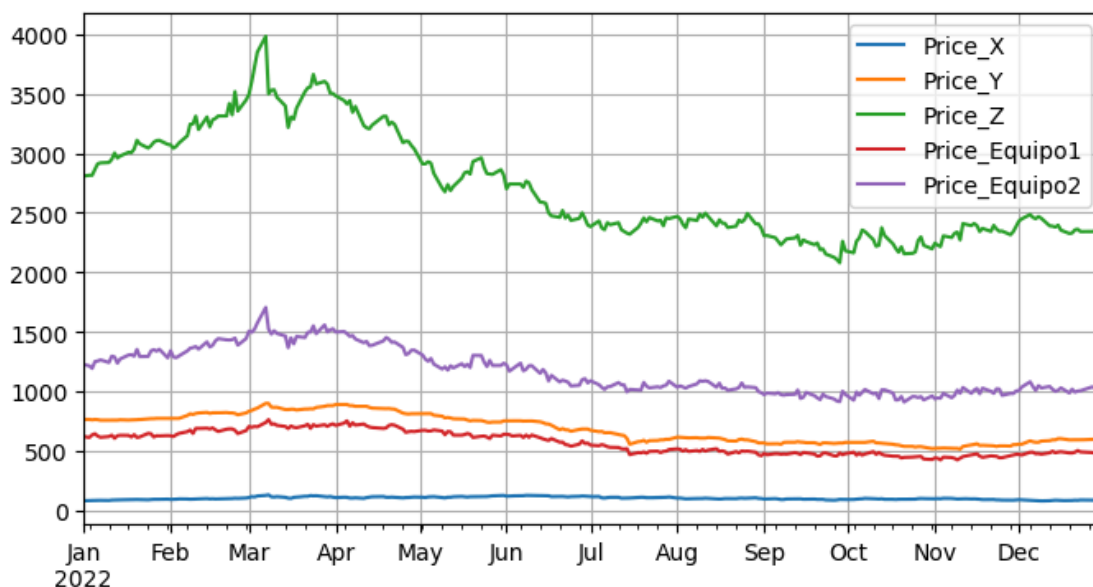
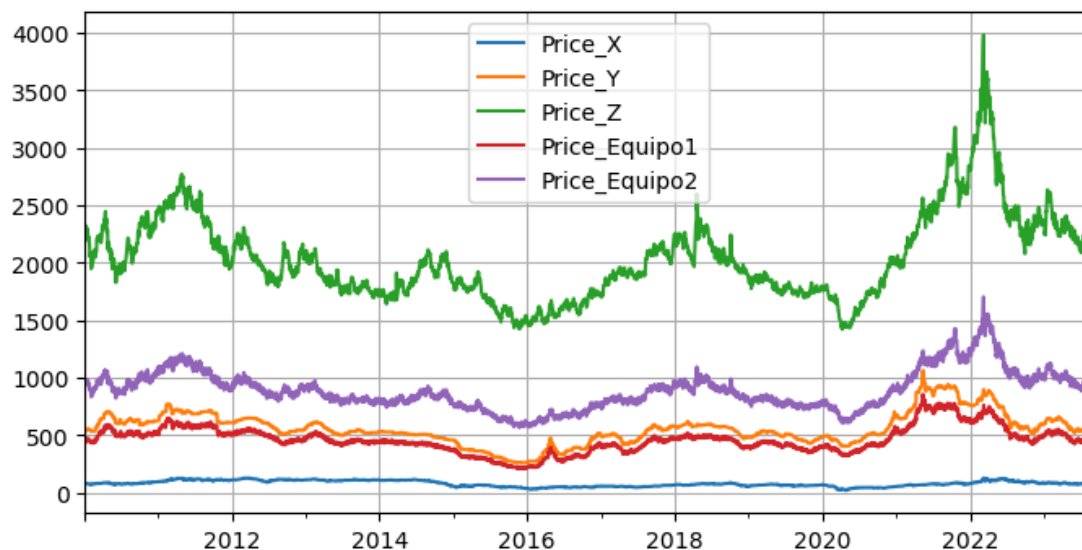


Figura 3. Gráfica de 2022 con valores imputados mediante interpolación lineal.

Después de esta transformación, podemos tener una vista más completa del comportamiento del pasado en una sola gráfica:



Antes de graficar e inspeccionar visualmente las distribuciones de cada variable, podemos computar métricas que capturen características de la distribución:

Skewness: Medida de asimetría de los datos, inclinándose a la derecha o izquierda.

Skewness of **Price_X**: 0.08224821266435678

Skewness of **Price_Y**: 0.4768885382027185

Skewness of **Price_Z**: 1.1451647384348107

Skewness of **Price_Equipo1**: 0.4059688020618139

Skewness of **Price_Equipo2**: 0.8412790272479774

Kurtosis: Medida de qué tan gruesas son las puntas de la distribución.

Kurtosis of **Price_X**: -1.1453946686916705

Kurtosis of **Price_Y**: 0.7271416283793326

Kurtosis of **Price_Z**: 2.042617219831026

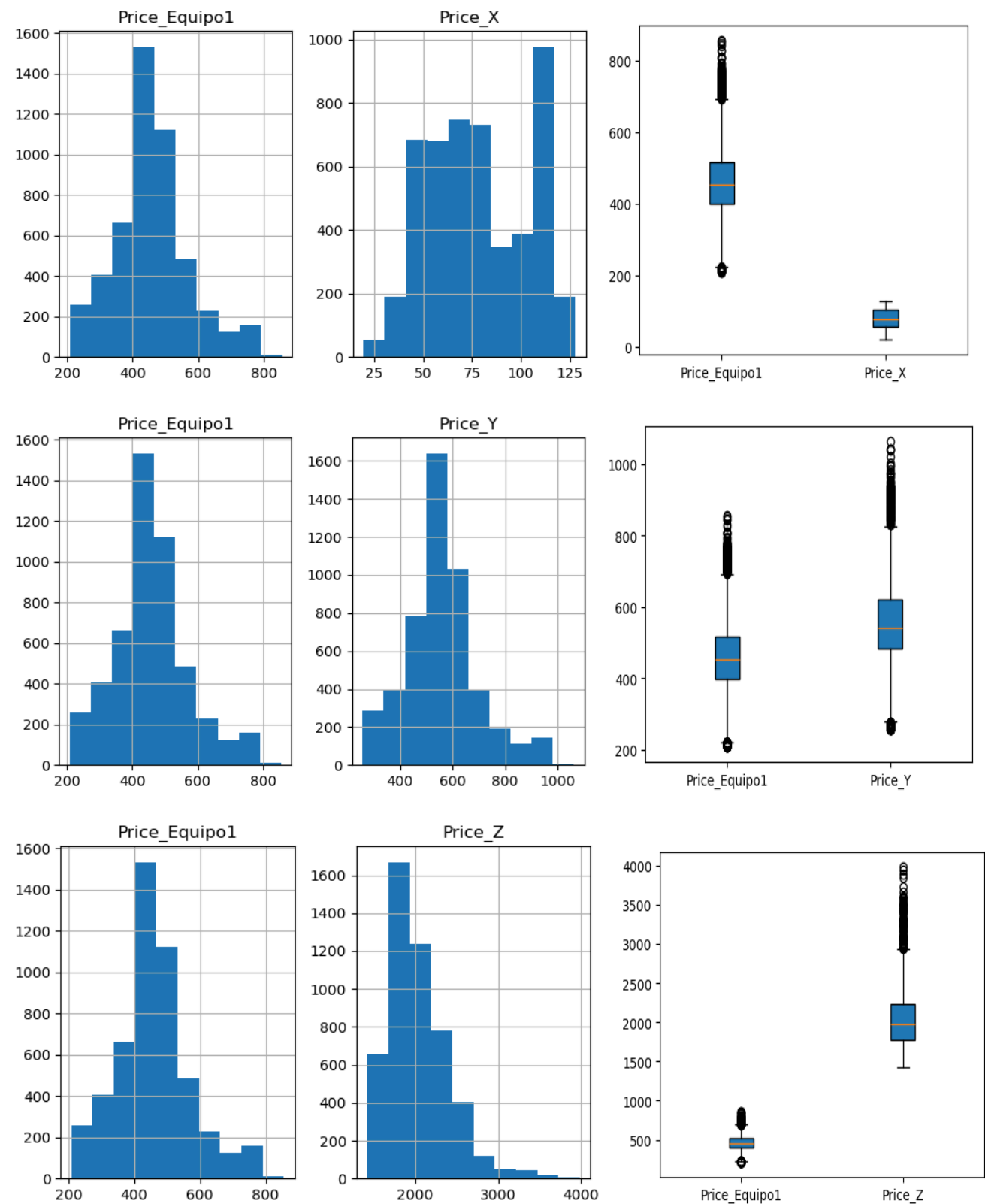
Kurtosis of **Price_Equipo1**: 0.582845454185462

Kurtosis of **Price_Equipo2**: 1.104114702865207

Ahora se puede pensar en comparar cada candidato de variable independiente/feature contra cada variable dependiente/target, comparando gráficamente la distribución por medio de histogramas y gráficos de caja o boxplots:

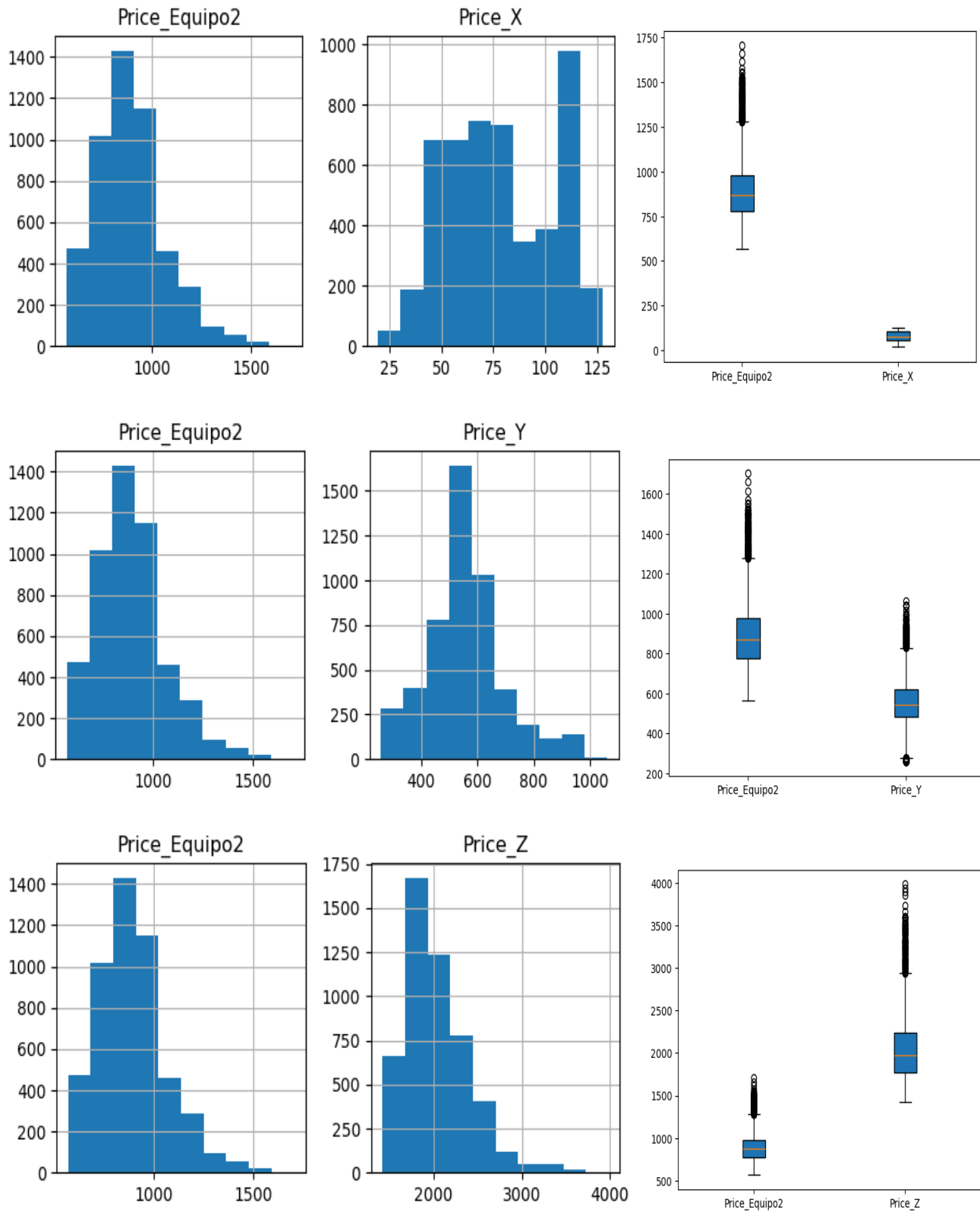
Equipo 1 vs. Insumos

Aqui podemos observar una similitud muy importante entre Equipo 1 y Y

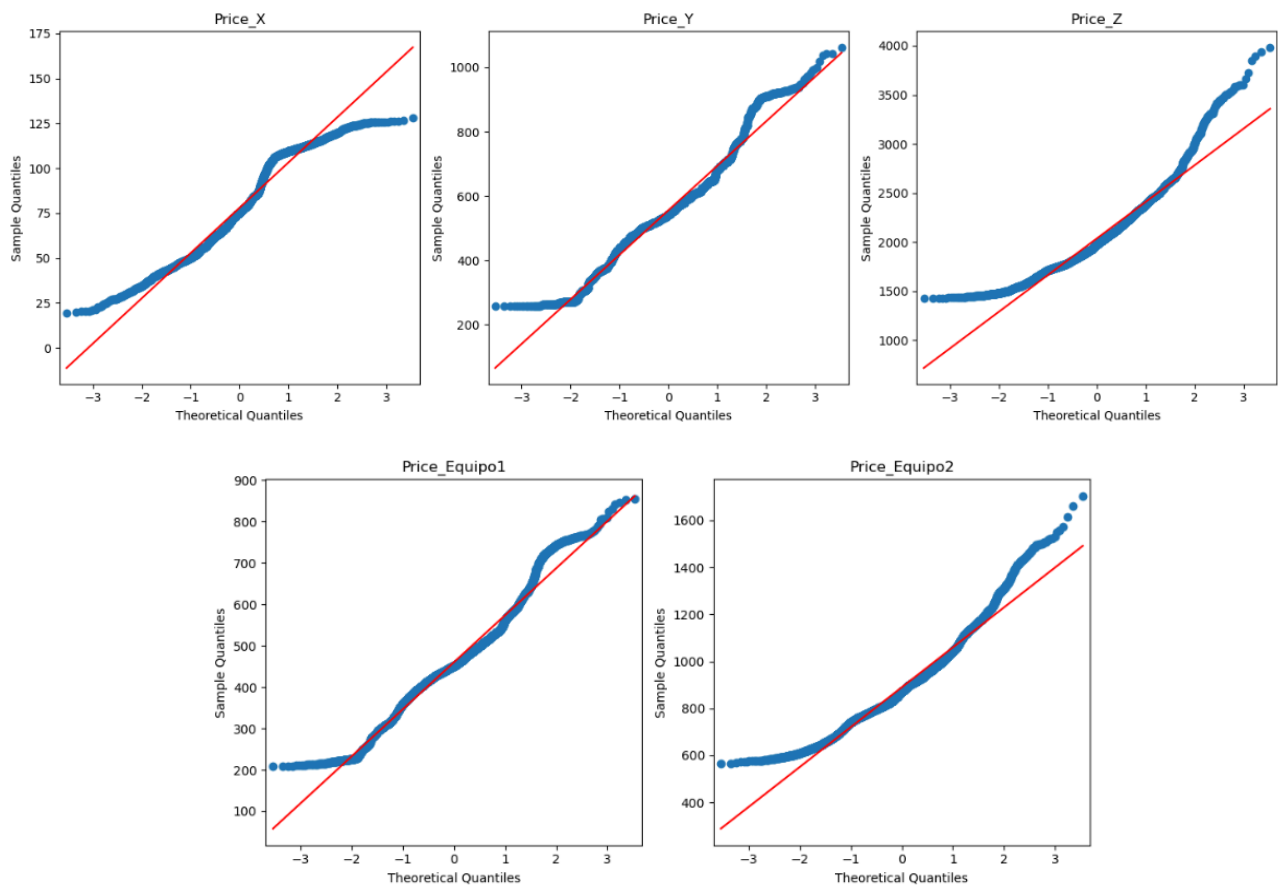


Equipo 2 vs. Insumos

Aqui podemos observar una similitud importante entre Equipo 2 y Z

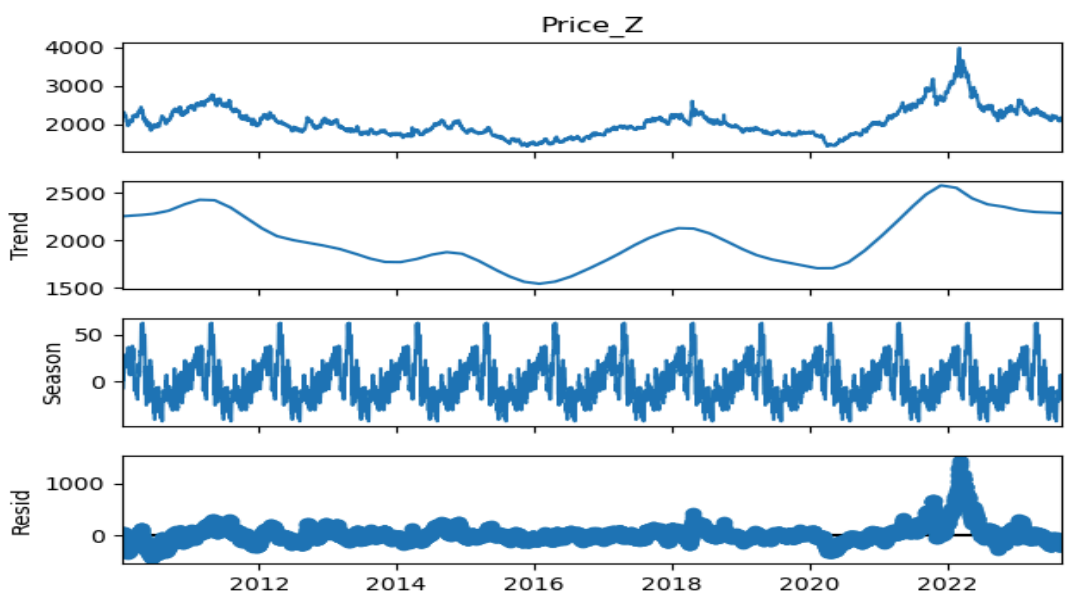
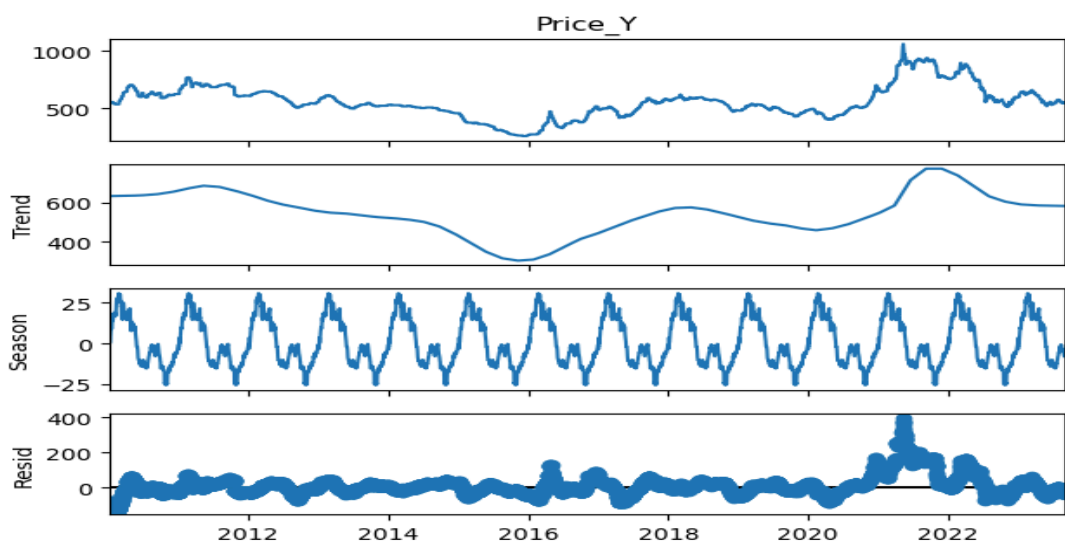
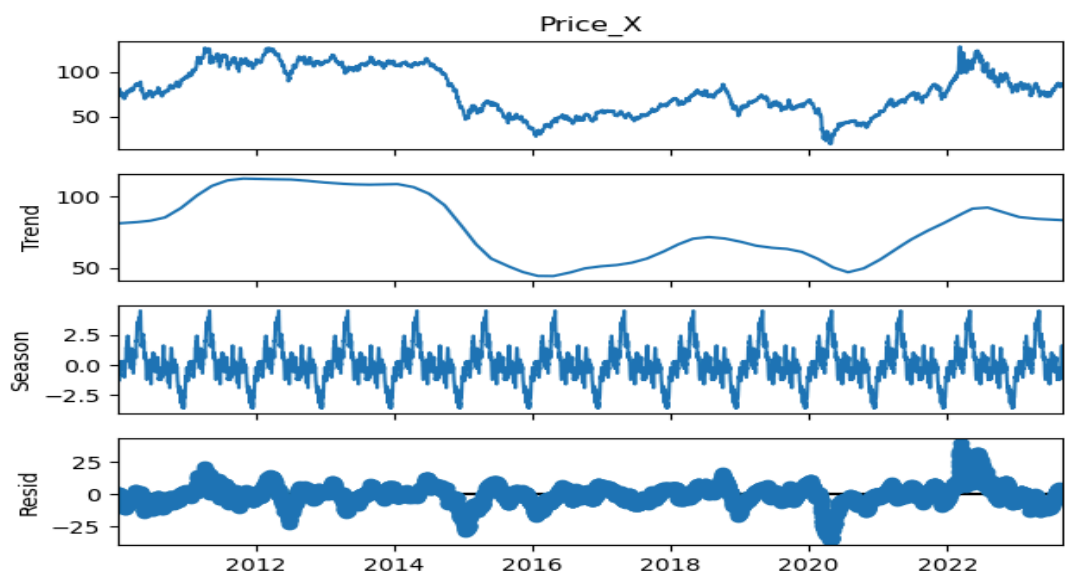


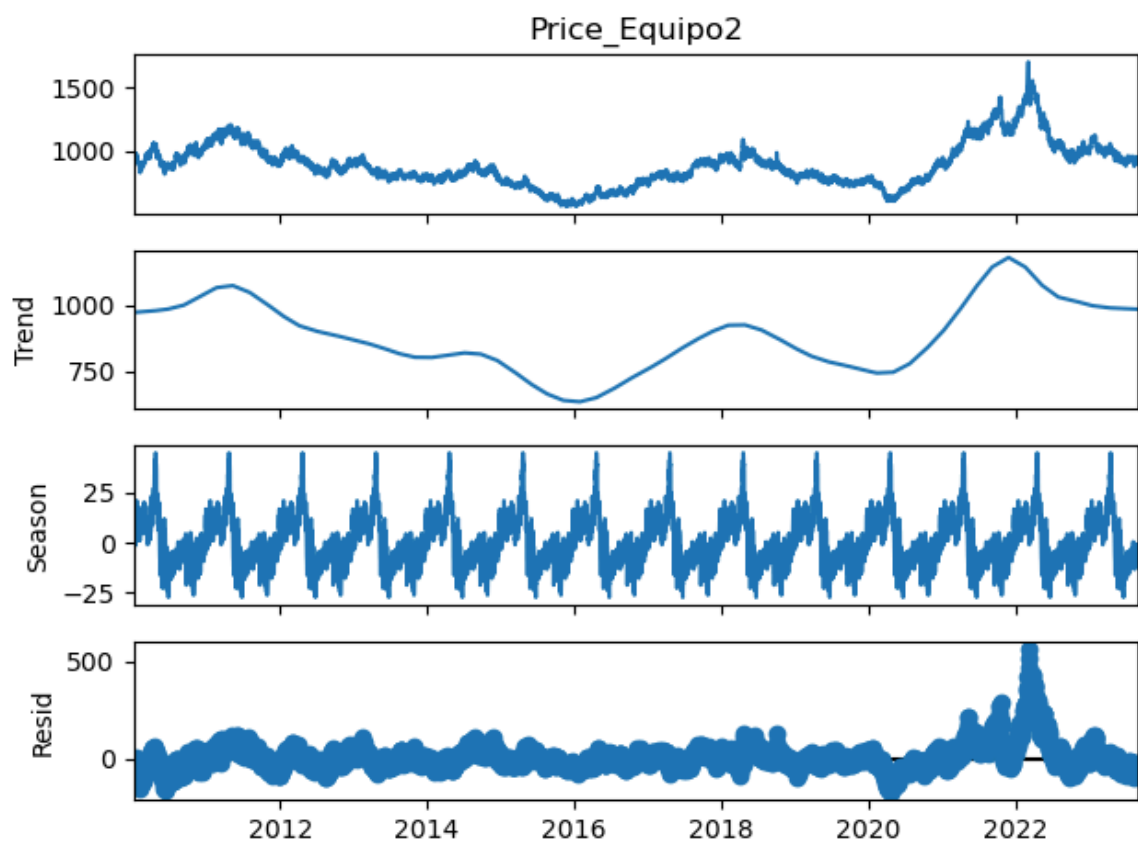
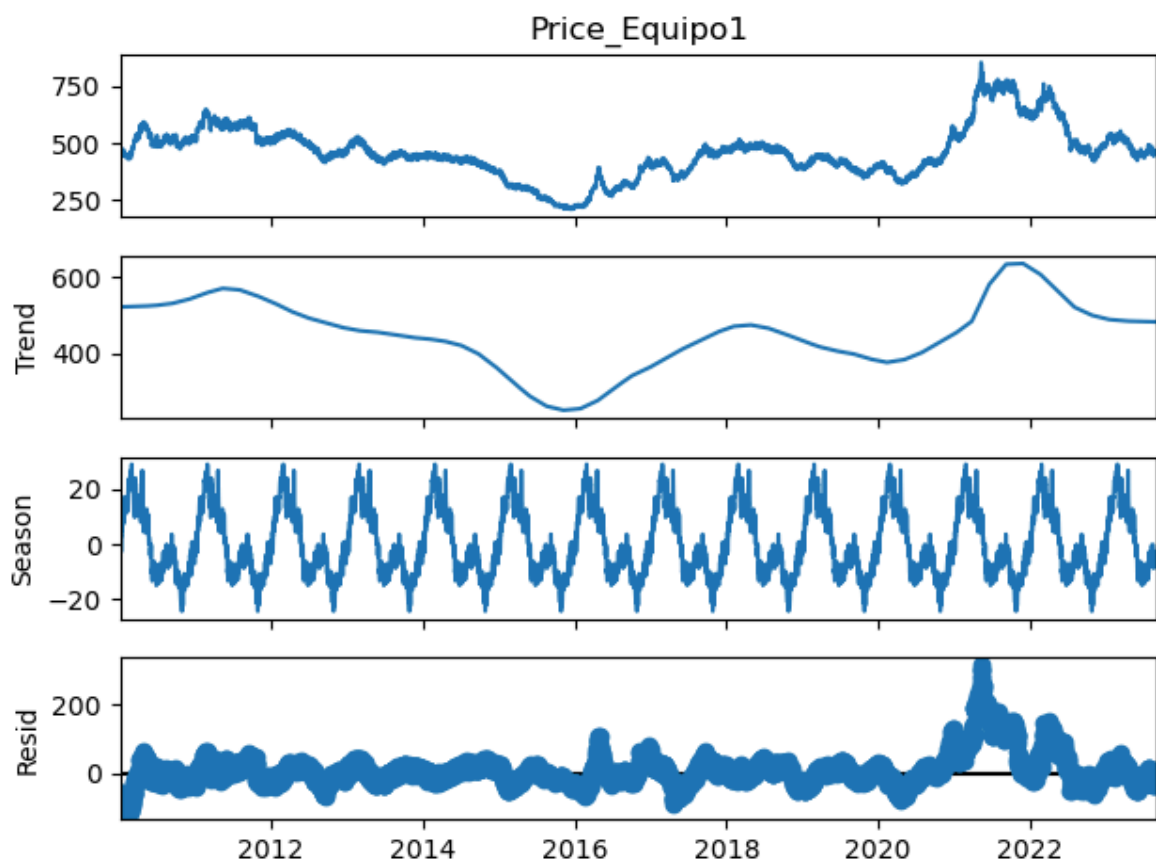
Q-Q plots: qué tan alineada está cada variable a una distribución normal teórica.



Descomposición estacional de cada variable

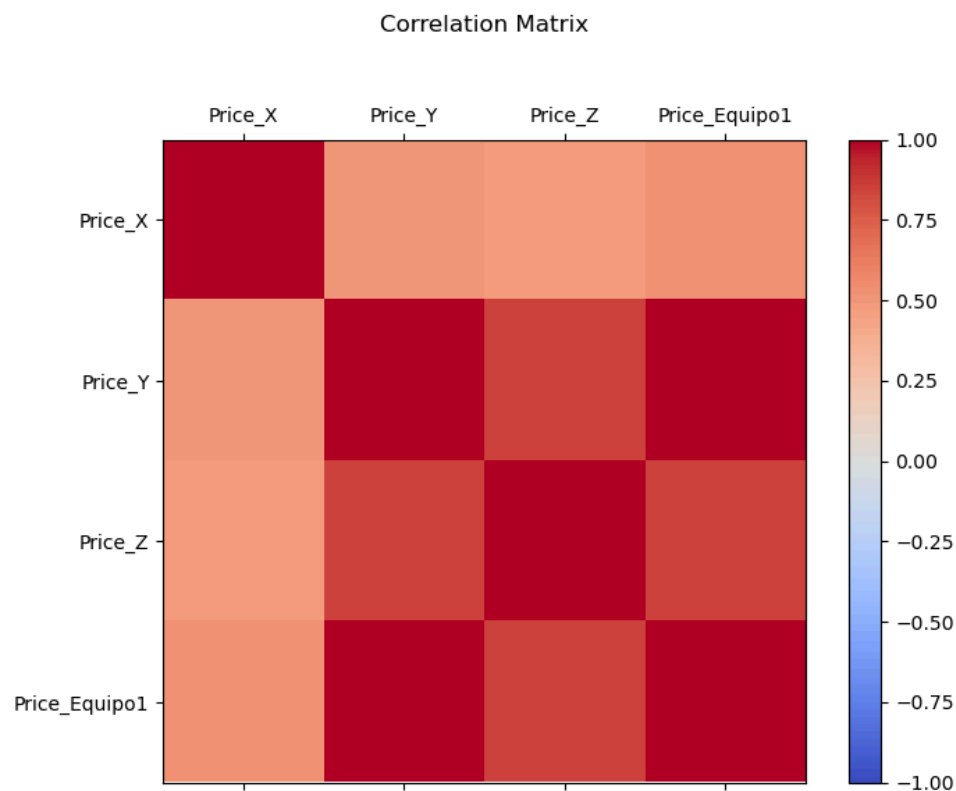
Finalmente, podemos ver si al descomponer la serie usando un algoritmo estadístico llamado **STL** (Seasonal and Trend decomposition using LOESS). Podemos ver que al usar un tamaño de ciclo de 1 año = 365 días, los datos históricos de cada variable muestran un componente estacional.



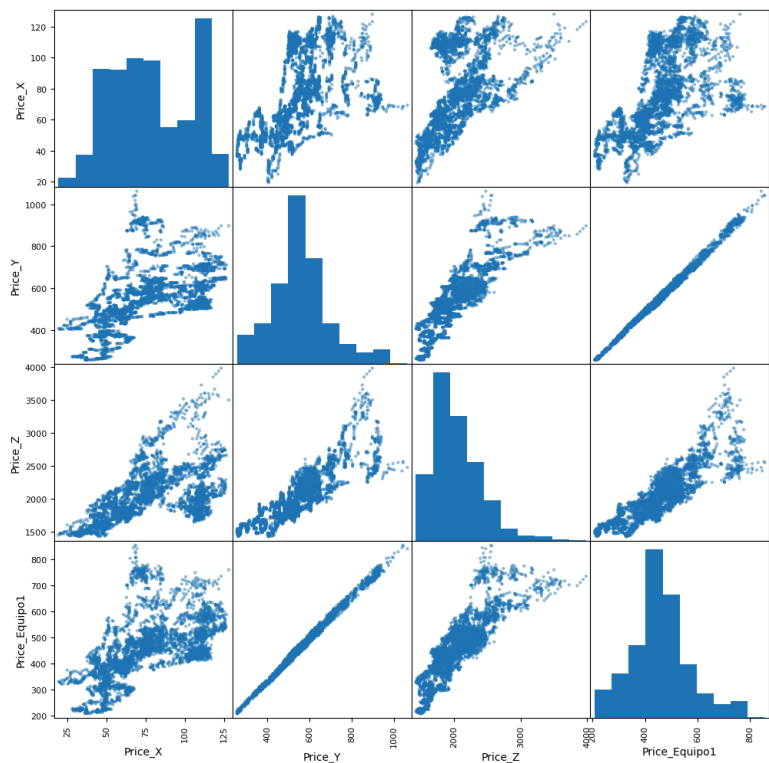


Análisis de correlación

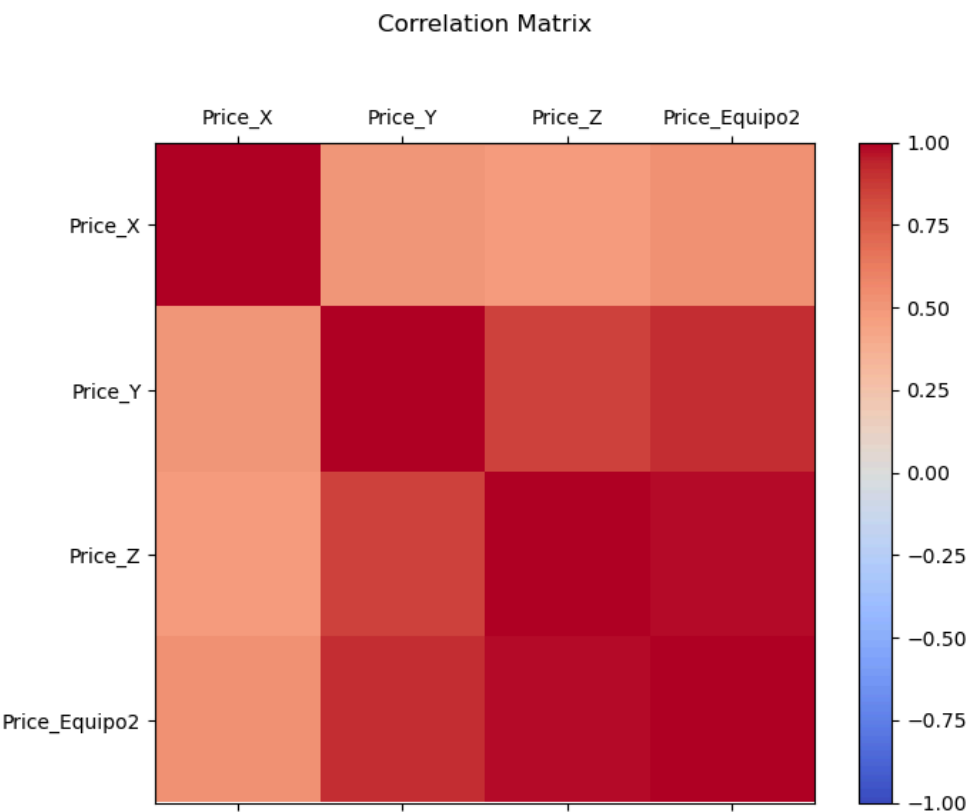
Equipo 1 VS Features



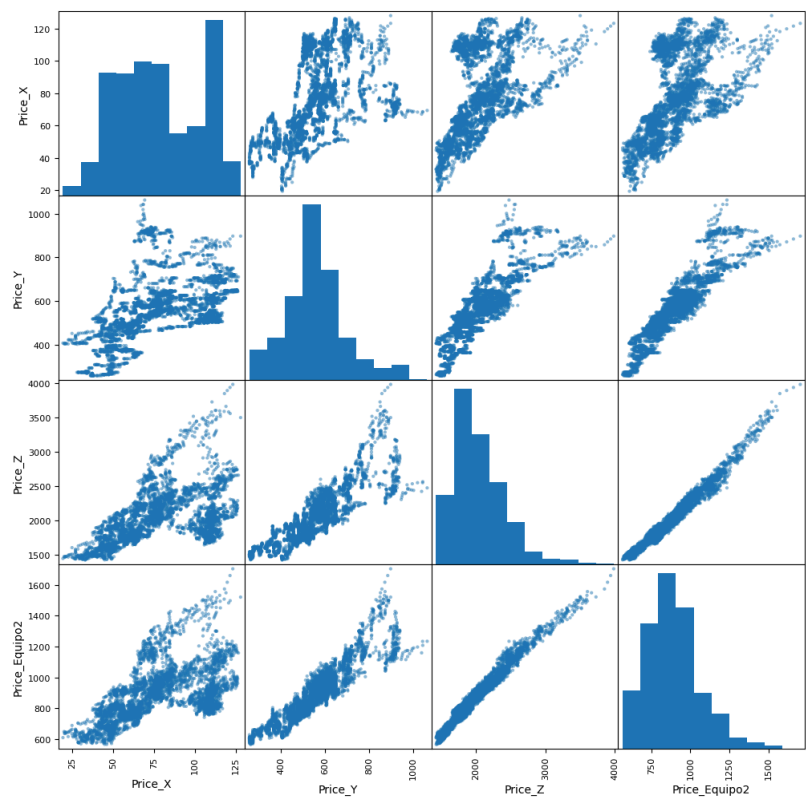
Como sospechaba y ahora se corrobora gráficamente, hay una correlación lineal fuerte entre Equipo 1 e Y, pero no menos importante es también la correlación de Equipo 1 y Z.



Equipo 2 vs. All Features



De manera similar, hay una correlación lineal fuerte entre Equipo 2 y Z, pero no menos importante es también la correlación de Equipo 2 e y.



Feature engineering

Una vez entendidas las características de cada modelo, se procede a crear features que ayuden a explicar el comportamiento futuro y estén disponibles también en ventanas futuras.

Los tipos de features que se busca generar y que sabemos tendremos en el futuro cuando aún no se conocen los precios para los equipos e insumos son:

- **Características temporales:** día, mes, año, día del año.
- **Features lags del target,** el valor del día anterior, hace una semana, hace un mes, hace un año, y con recursión por parte del modelo.
- **Features lags de los insumos,** cuáles han sido los precios de los insumos en el pasado, hace 7, 15, 30 o 365 días; estos tipos de lags dependen del tamaño de la ventana de tiempo futura.
- **Componentes estacionales de cada variable:** Ya que se comprobó un fuerte componente de ciclo anual para cada variable, se usa como feature futuro el valor del componente de estacionalidad para ese día a predecir.

Dado que usaremos modelos regresores basados en árboles de decisión, no es necesario escalar o transformar los features o targets, por lo pronto. En futuras mejoras y en otros tipos de modelos a probar, sí sería necesario. Por lo pronto, se cuenta con la siguiente lista de features aumentados generados automáticamente en función de la variable **future_horizon**:

Características disponibles en entrenamiento y en future_horizon

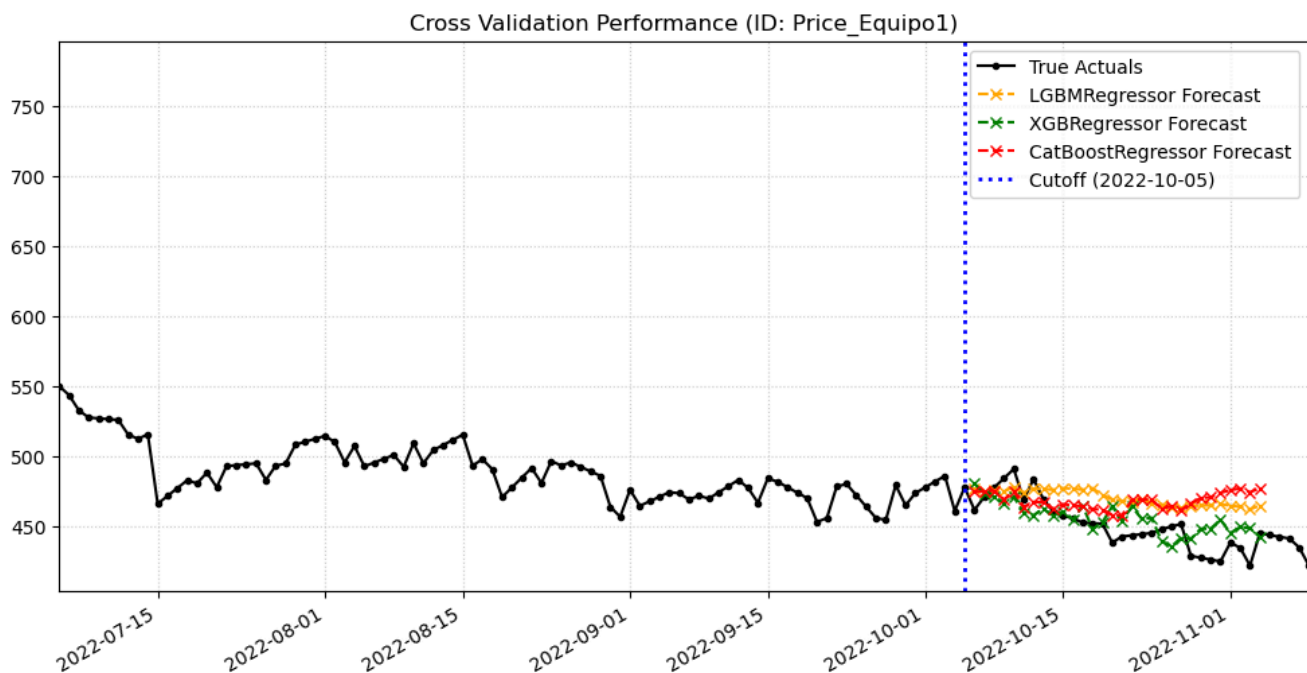
- **Price_Y_lookback_365**
- **Price_Z_lookback_365**
- **seasonal_Price_Y**
- **seasonal_Price_Z**
- **seasonal_Price_Equipo1**
- **seasonal_Price_Equipo2**
- **[lag1, lag7, lag30, lag365]**
- **[year, month, dayofweek, dayofyear]**

Cross-validation

Si tenemos en cuenta que tenemos más de 13 años de datos de frecuencia diaria y también queremos poder predecir en esta frecuencia, sin que la incertidumbre futura o errores en multi-step forecast se acumulen en largas ventanas de tiempo, esto significa que tenemos una oportunidad de hacer un gran número de validaciones en la historia en múltiples puntos, y así ver cuál modelo logra predecir mejor en general.

Usamos un valor por default de 20 ventanas de validación del tamaño de **future_horizon**. Suponiendo que se usaran frecuencias futuras diarias como [7, 15, 30, 60]. Si las ventanas fueran a ser mayores, lo cual es posible pero incrementa la incertidumbre, este número se puede reducir un poco, para obtener tiempos de validación más rápidos y cuidando tener suficiente data de entrenamiento y validación en cada ventana de simulación.

Un ejemplo de validación hacia atrás en series de tiempo con múltiples modelos, se visualiza de la siguiente manera:



Con los modelos candidatos en colores, los cuales entrenaron con data truncada hasta la fecha **cutoff**. Los valores reales posteriores a esta fecha pueden ser comparados con data real y así medir diferentes errores para esta ventana. Posteriormente, se usa alguna estrategia de agrupación para resumir el performance en general de cada modelo.

Resultados del análisis de: Modelos

Para future_horizon=30 y 20 ventanas de validación, unos cuantos registros de error modelo SMAPE para cada modelo.

unique_id	cutoff	metric	LGBMRegressor	CatBoostRegressor	XGBRegressor
Price_Equipo1	2022-01-08	smape	0.90%	3.19%	1.23%
Price_Equipo2	2022-01-08	smape	3.77%	5.78%	7.92%
Price_Equipo1	2022-02-07	smape	4.05%	1.93%	2.70%
Price_Equipo2	2022-02-07	smape	4.06%	5.78%	3.85%
Price_Equipo1	2022-03-09	smape	8.12%	3.34%	4.11%
Price_Equipo2	2022-03-09	smape	2.06%	1.79%	1.39%
Price_Equipo1	2022-04-08	smape	4.02%	2.79%	0.97%
Price_Equipo2	2022-04-08	smape	4.90%	3.65%	3.37%
Price_Equipo1	2022-05-08	smape	3.30%	1.90%	3.27%
Price_Equipo2	2022-05-08	smape	1.27%	2.03%	1.15%
Price_Equipo1	2022-06-07	smape	4.53%	3.74%	4.07%
Price_Equipo2	2022-06-07	smape	4.36%	4.42%	3.23%

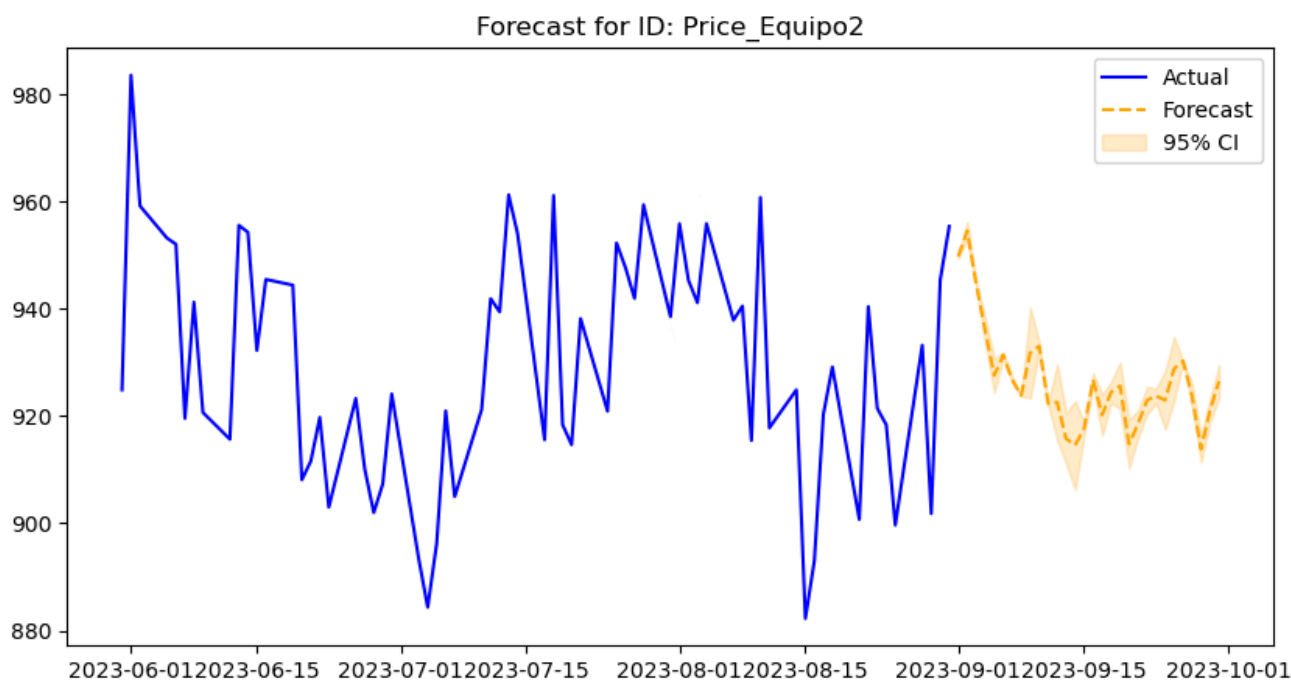
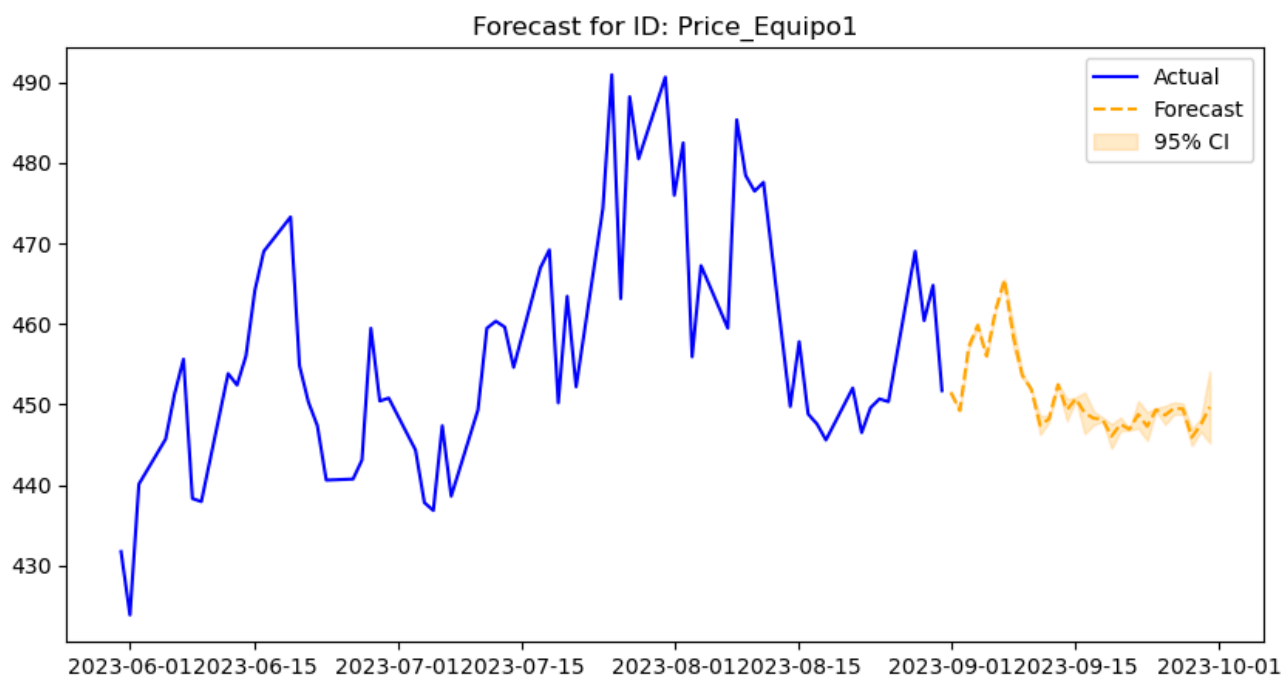
Luego, con esta tabla se elige el mejor modelo que, en este caso, viendo todos los registros de error por cutoff, el modelo ganador es XGBRegressor con las siguientes métricas:

Metric	Metric_value
SMAPE	2.41%
MAPE	4.80%
RMSE	47.24
MAE	40.6

Horizonte de predicción

Esta metodología de creación de modelo se ajusta y optimiza con base a la necesidad de proyección al futuro del negocio, por lo que no se define una ventana fija, sino dinámica.

Predicción futura



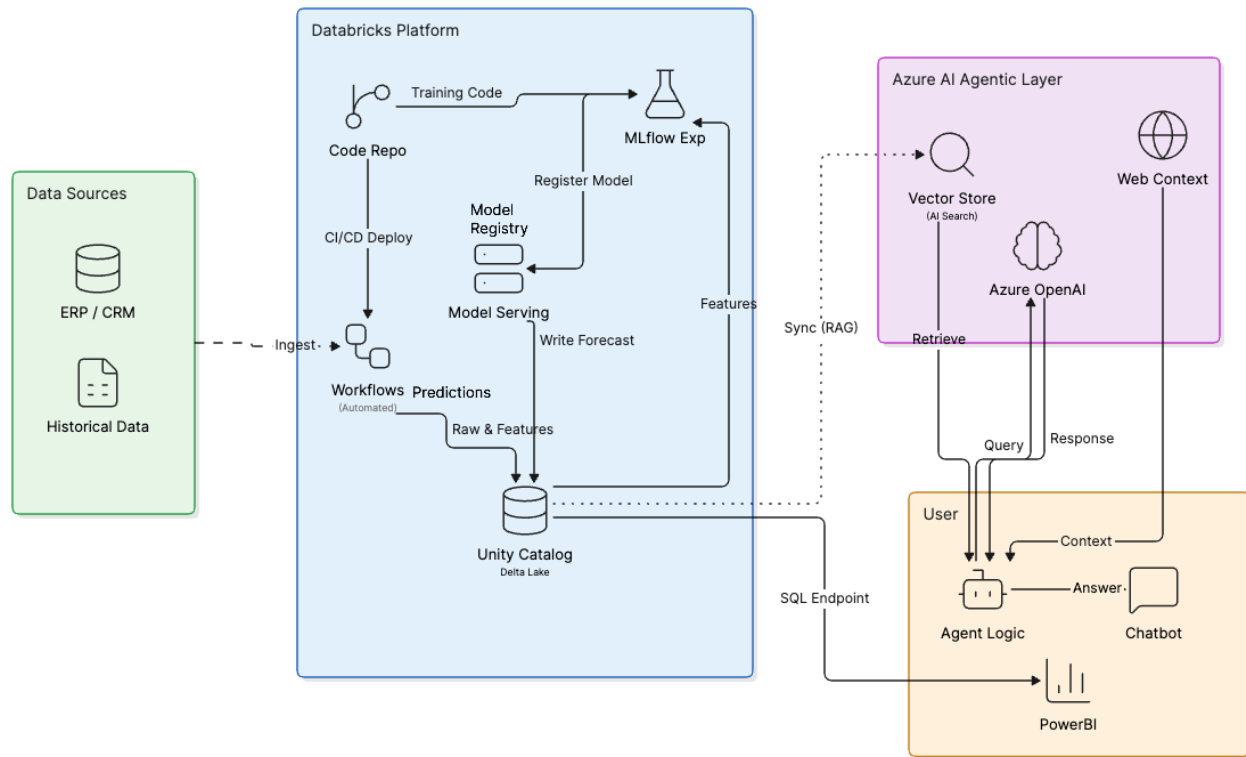
Futuros ajustes o mejoras

- Hiperparametrización. Para cada modelo se puede pasar un objeto de Optuna que será encargado de buscar el mejor conjunto de hiperparámetros y buscar el mejor desempeño.
- Se puede agregar o evaluar una gran variedad de features como transformaciones, trends y fast Fourier transform que puedan aumentar el poder predictivo, sin caer en el problema de alta dimensionalidad.
- Se puede evaluar en la pipeline de evaluación si un modelo dedicado para cada equipo mejora sus resultados.
- Se podría recurrir a la estrategia de separar el problema en varios subproblemas, y modelar los componentes por separado, trend y noise, forecastear cada uno por separado y luego combinar los resultados.
- Se puede definir una estrategia de ponderación de ventanas Moving Average para selección de modelo, de esta forma, los resultados de la data más reciente tienen más peso que ventanas antiguas, pero que no dejan de aportar.

Apreciaciones y comentarios del caso

Por cuestiones prácticas y de tiempo no se abordan, pero tampoco se desconocen, técnicas más avanzadas en cuanto a feature engineering, feature selection, hiperparametrización o finetuning de modelos. Así como el uso y combinación de modelos estadísticos, deep learning o modelos fundacionales preentrenados multipropósito basados en transformadores para data secuencial.

Arquitectura cloud propuesta: Azure



Esta arquitectura propone transformar los modelos aislados o desarrollados en local en una plataforma de forecasting productiva y escalable, integrando MLOps e insights generados con interfaces conversacionales de IA.

Componentes de la arquitectura productiva

- **Capa de Data Science & MLOPS**
 - **ERP y fuentes:** conexión a las fuentes de datos internos
 - **Experimentación y MLOps:** notebooks, experimentos replicables
 - **Gestión:** Repositorio (Git) para versiones y colaboración.
 - **Orquestación:** Workflows y jobs, para automatizar ingestas, reentrenamiento e inferencias batch.
 - **Gobierno de datos:** Unity Catalog como fuente única de verdad (datos, feature store y predicciones).
 - **Despliegue:** Model Serving para exponer el modelo vía API REST, permitiendo inferencia de baja latencia.

- **Capa Inteligente** (Azure AI) despliegue, fuentes de conocimiento, configuración de modelos finetuneados y prompt engineering de cara a un agente conversacional.
 - **Modelo: Azure OpenAI:** Genera respuestas en lenguaje natural, interpreta intenciones y sintetiza información técnica. Podría ser otro modelo open source o de pago.
 - **Vector Store:** Almacena contexto semántico (documentación, histórico, reglas de negocio) para habilitar Retrieval-Augmented Generation (RAG).
 - **Web Context:** Enriquecimiento con datos de mercado, tendencias y noticias en tiempo real.
- **Capa de consumo** (interfaces) formas de usar los insights generados.
 - **Chatbot:** Acceso conversacional para usuarios de negocio sin requerir conocimientos técnicos. Puede ser a través de Teams o una web app.
 - **Power BI** Dashboards para analítica visual, KPIs y monitoreo de precisión.

Lógica de agente

Este es básicamente el componente crítico que implementa el patrón ReAct que se basa en razonar y actuar; sus funciones son:

- **Clasificación de intención:** Entiende si el usuario busca un forecast, una explicación de tendencia o una comparación.
- **Ensamblaje de contexto:** Consulta el Vector Store, el Web Context y el Unity Catalog para construir prompts con datos precisos y no alucinar valores o caer en ciclos de pregunta del usuario sobre información de la cual no se tiene.
- **Ingeniería de prompts:** Define el rol del asistente, aplica guardrails para evitar alucinaciones y formatea respuestas.
- **Orquestación de herramientas:** Ejecuta llamadas a APIs, gestiona el estado de la conversación y recupera información de múltiples fuentes.

Al final de cuentas, se busca que esta solución no solo entregue una predicción numérica, sino que provea explicaciones coherentes y fundamentadas en el contexto de negocio, manteniendo la trazabilidad desde la ingesta de datos hasta la respuesta final.

Diferencia entre agente de IA y sistema de IA convencional

Un sistema de **IA convencional** es un motor de procesamiento pasivo de datos, un **agente de IA** es un sistema activo diseñado para alcanzar objetivos mediante la interacción con su entorno. En el contexto de chatbots, el entorno es la información conversacional con los usuarios.

Modelo ML tradicional

Un modelo de IA convencional (como un regresor o un clasificador) actúa como una función matemática compleja: recibe una entrada (*input*), la procesa a través de parámetros aprendidos y devuelve una salida (*output*), sin tomar acciones más allá de su respuesta numérica o categórica.

Agente de IA

Un agente de IA es un sistema autónomo que integra modelos de lenguaje (como un cerebro central) con una arquitectura que le permite percibir, razonar y actuar.

- **Autonomía:** Un agente puede descomponer un objetivo de alto nivel en múltiples subtarefas de forma independiente.
- **Uso de herramientas:** El agente tiene la capacidad de invocar funciones externas. Puede ejecutar código, consultar una base de datos SQL, realizar búsquedas web o invocar APIs para obtener datos que no están en su memoria de entrenamiento. Incluso interactuar con otros agentes y automatizaciones mediante protocolos estándar como MCP.
- **Memoria:** Posee tanto memoria de corto plazo (historial de la conversación) como memoria de largo plazo que puede ser consultada mediante una técnica como RAG, lo que le permite recordar el contexto de interacciones previas y documentos específicos.
- **Capacidad de acción:** Un modelo de ML tradicional solo "predice", un agente puede "actuar". Esto implica la capacidad de modificar estados en otros sistemas.